

RELAZIONE PROGETTO

“CONNECTIONS”

Candidato: Giacomo Giarelli

Istruzioni per l'uso

Il sistema richiede Java 8 o superiore. Le dipendenze esterne (Gson 2.13.2) sono incluse nella cartella “lib”, e non sono richiesti argomenti da linea di comando: le impostazioni sono caricate automaticamente dalla cartella “resources” dove risiedono file Properties per client e server. Il programma è stato sviluppato e testato in ambiente Windows.

Compilazione: per la compilazione del codice sorgente è sufficiente questo comando:

```
javac -cp "lib/*" -d bin src/common/*.java src/server/*.java src/client/*.java
```

Esecuzione: Aprire il terminale nella cartella del progetto ed eseguire:

Server: *java -cp "Server.jar;lib/*" server.ServerMain*

Client: *java -cp "Client.jar;lib/*" client.ClientMain*

I file eseguibili sono stati già forniti, ma si possono rigenerare attraverso l'utility *jar*.

Il Client è dotato di un menù interattivo dinamico, dove l'utente può selezionare le operazioni digitando il numero corrispondente (es. 1 per la registrazione, 2 per il login). In base allo stato (autenticato o meno), il menù mostrerà solo le azioni disponibili.

```
--- MENU ---
Scegliere un numero per svolgere la relativa funzione.
1 - Registra un nuovo utente
2 - Login
3 - Disconnetti dal server
```

Architettura del server

1. Architettura di Rete

Il server è basato su un Selector in modalità non bloccante eseguito sul thread principale, il cui compito è gestire il multiplexing delle connessioni: accettare nuovi client, monitorare i canali per la lettura e coordinare l'invio delle risposte.

Quando il Selector rileva dati in ingresso, ricostruisce l'intero messaggio JSON che potrebbe arrivare frammentato in più segmenti TCP; questo avviene grazie a un protocollo di comunicazione che prevede un header di 4 byte che indica la dimensione del pacchetto. Una volta completata la ricezione, il messaggio viene inoltrato a un `CachedThreadPool`. Questo pool di worker si occupa del parsing e dell'elaborazione delle richieste delegando poi la fase di invio della risposta nuovamente al thread principale del Selector, garantendo così che l'I/O sia gestito in modo centralizzato.

I messaggi di risposta contengono, di base, un campo *operation* e un campo *err_code*: il primo specifica a quale operazione appartiene la risposta; il secondo indica l'esito dell'operazione che verrà poi stampato dal Client.

A ogni `SocketChannel` viene associato un oggetto `Session` tramite il meccanismo degli attachment. Questo oggetto è fondamentale per gestire la lettura frammentata, sia perché accumula i byte ricevuti fino alla completa ricostruzione della stringa JSON, e sia perché associa l'identità di un utente loggato al canale. In questo modo, ogni operazione successiva viene riferita automaticamente all'utente legato al canale.

Oltre alla connessione TCP persistente, il server offre due canali di comunicazione supplementari:

- **Registrazione (RMI):** un servizio dedicato su una porta separata per l'iscrizione dei nuovi utenti, isolando questa procedura dal traffico di gioco principale.
- **Notifiche Asincrone (UDP):** due sistemi di notifica basati su Multicast e Unicast. Il primo invia datagrammi per notificare la fine di una partita e aggiornare la classifica, il secondo invia a ogni utente le proprie statistiche quando una partita termina per timeout.

I messaggi di risposta includono un codice di errore specifico che permette al client di distinguere tra un'operazione andata a buon fine e diverse tipologie di

fallimento (es. credenziali errate, partita conclusa o errori di validazione). Infine, le impostazioni sono configurabili esternamente tramite il file `server.properties`, permettendo una manutenzione agevole senza intervenire sul codice sorgente.

2. Logica di gioco

La logica di gioco è gestita dall'oggetto `GameManager`, il cui compito è occuparsi dell'intero set di partite e coordinarne il ciclo di vita. Il sistema è progettato per evitare di mantenere l'intero database delle partite in memoria.

Invece di caricare l'intero set di dati all'avvio, il `GameManager` mantiene in memoria una lista composta dagli ID partita e dai relativi offset all'interno del file sorgente JSON. Questo approccio permette di ridurre l'occupazione di memoria, reperire le informazioni di una specifica partita tramite l'offset creando dinamicamente l'oggetto `GameSession` solo quando necessario, e infine salvare nel file `server.properties` l'indice dell'ultima partita disputata, consentendo al server di riprendere la sequenza corretta dopo un riavvio.

Inizialmente il `GameManager` viene lanciato dal thread principale del server, in cui carica la lista di offset e la prima partita della sessione. L'esecuzione del `GameManager` viene poi affidata a uno `ScheduledExecutorService`, il quale esegue questo ciclo:

1. **Chiusura:** Il thread disattiva la partita in corso alla scadenza del timeout.
2. **Classifica:** la classifica globale viene aggiornata considerando i punteggi della partita appena terminata.
3. **Unicast UDP:** I giocatori vengono notificati delle statistiche individuali e collettive della partita appena terminata tramite Unicast UDP.
4. **Persistenza:** Salvataggio delle statistiche collettive e individuali delle partite e aggiornamento delle statistiche globali dei singoli giocatori (vittorie, partite totali, serie di vittorie...) su file persistenti. I dati vengono poi salvati su disco in modo atomico.
5. **Nuova partita:** Effettua il parsing del JSON di una nuova partita utilizzando la lista di offset e istanzia una nuova `GameSession`.
6. **Attivazione:** La partita viene resa disponibile per i client e inizia il countdown del timeout.

7. **Multicast UDP:** I giocatori vengono notificati della fine partita e viene chiesto loro se vogliono prendere parte alla nuova partita.
8. **Fase Attiva:** Il server gestisce le proposte dei client.

3. Persistenza e recupero

Il server utilizza due file JSON: `users.json` per le credenziali e le statistiche dei singoli utenti, e `games.json` per lo storico delle partite. All'avvio i dati vengono caricati in due `ConcurrentHashMap`, che garantiscono accessi rapidi e sicuri in ambiente multithread.

È stato implementato un sistema di sicurezza tramite il file `"users_temp.log"`. Ogni registrazione o cambio di credenziali viene scritto immediatamente in modalità *append* su questo file. In caso di crash, al riavvio il sistema riapplica in sequenza le operazioni del log alla `HashMap` degli utenti, ricostruendo lo stato aggiornato. Si è scelto di non applicare questo log alle partite in corso poiché le informazioni sugli utenti sono considerate prioritarie; inoltre, una partita è considerata valida solo se termina il suo intero ciclo di vita. In caso di interruzione forzata, la partita attiva viene scartata e ricaricata all'avvio successivo.

Al termine di ogni partita, il thread scheduled delega l'aggiornamento dei file alla classe `FileUpdater`. Per evitare corruzioni da eventuali crash, si adotta una strategia di sovrascrittura atomica: i dati vengono serializzati su un file temporaneo (es. `users_new.json`) e, solo a scrittura completata, si utilizza il metodo `Files.move()` con l'opzione `ATOMIC_MOVE` per sostituire il vecchio file.

4. Strutture dati e sincronizzazione

Per garantire l'efficienza e la sicurezza multithread, sono state utilizzate le collezioni della libreria `java.util.concurrent`:

- **Gestione Utenti:** Una `ConcurrentHashMap<String, User> users` permette accessi in tempo costante per le ricerche e le operazioni sugli utenti.
- **Sessione di Gioco Attiva:** Durante la partita, la `GameSession` utilizza una `ConcurrentHashMap<Integer, PlayerGameStats> playerStats` per

consentire ai thread del pool di aggiornare e consultare simultaneamente i punteggi dei partecipanti.

- **Archivio Storico:** Le partite concluse sono inserite in una **`ConcurrentHashMap<Integer, GameStats> gameStats`**. In questa fase di transizione, la mappa interna delle statistiche individuali viene convertita in una semplice `HashMap`: poiché i dati storici sono consultabili solo in lettura, l'assenza di sincronizzazione elimina l'overhead senza rischi di corruzione.
- **Classifica Globale:** Modellata come un `ArrayList<User>`, viene aggiornata atomicamente sostituendo l'intero riferimento alla lista. Questo garantisce ai thread un riferimento attendibile e permette di sfruttare il metodo `sort()` (ordinamento per punteggio e per ordine alfabetico).

Meccanismi di Sincronizzazione

Il server sfrutta l'atomicità implicita delle operazioni sulle mappe per evitare blocchi espliciti e utilizza delle variabili `Atomic` per le statistiche collettive della partita (vincitori, partecipanti totali), garantendo atomicità nelle modifiche e nelle letture. E' stata utilizzata una `ReadWrite Lock` per accedere alla sessione di gioco attiva; per alcune operazioni, infatti, è fondamentale che la partita sia disponibile. Questo approccio permette a più client di accedere alla sessione attiva concorrentemente garantendo che la sessione non venga disattivata nel mentre.

Architettura del client

Il comportamento del client è modellato su una macchina a stati finiti (stati LOGGED e NOT_LOGGED). Questa distinzione permette di filtrare dinamicamente le operazioni concesse, adattando il menù interattivo e i comandi allo stato dell'utente.

L'interazione avviene tramite un'interfaccia testuale che guida l'utente all'uso delle funzioni messe a disposizione.

Il client interagisce con il server sfruttando tre modalità di comunicazione distinte:

- **RMI:** utilizzata esclusivamente per la registrazione dei nuovi utenti, isolando la logica di creazione account dal traffico di gioco.
- **TCP:** il canale primario per lo scambio di messaggi JSON, utilizzato per l'autenticazione, l'invio delle proposte e il recupero delle statistiche.
- **UDP :** due canali passivi, uno Multicast e uno Unicast, per la ricezione di aggiornamenti globali sulla classifica, notifiche di fine partita, statistiche individuali e collettive di ogni partita al suo termine.

L'architettura del client si appoggia su tre thread. Il thread principale è responsabile dell'intero ciclo di interazione e gestisce la logica di input/output, interazione con l'utente, la serializzazione delle richieste e il parsing delle risposte.

Un secondo thread resta costantemente in ascolto su un indirizzo Multicast e alla ricezione di un datagramma non interagisce direttamente con l'interfaccia, ma imposta un flag apposito, condiviso col thread principale.

Un terzo thread resta in ascolto per il traffico UDP Unicast, da cui riceverà datagrammi contenenti statistiche individuali e collettive di partite che terminano al timeout. Come il thread precedente, imposta un flag apposito condiviso col thread principale.

Per evitare di interrompere l'input dell'utente sul terminale in momenti inopportuni, il thread principale controlla lo stato di tali flag solo in due momenti precisi: prima della stampa del menù e subito dopo l'interazione con esso.

Logica di Visualizzazione e Parsing

La gestione dell'output è affidata alla classe `Printer`, controllata esclusivamente dal thread principale. Tale classe centralizza la logica di parsing dei messaggi JSON e la loro stampa a video. Alla ricezione di una risposta dal server, il thread principale smista la richiesta verso il metodo opportuno della classe `Printer` basandosi sul campo *operation* presente nel messaggio JSON. Oltre alla tipologia di operazione, ogni messaggio include un campo *err_code*: un codice di stato che rappresenta l'esito dell'operazione e permette alla classe `Printer` di fornire un feedback immediato e specifico all'utente (es. successo, errore di autenticazione, input non valido).

Strutture dati e sincronizzazione

L'architettura del client è stata progettata per minimizzare l'uso di memoria e delegando la persistenza dei dati al server.

Il client non memorizza collezioni di dati complesse. Le uniche informazioni mantenute in memoria sono l'utente autenticato e l'ID della partita corrente. I dati ricevuti dal server vengono processati e stampati senza essere memorizzati, utilizzando lo streaming API della libreria `Gson`.

La sincronizzazione lato client è limitata alla gestione dei flag di notifica per i pacchetti UDP. Questi flag sono stati implementati come `AtomicBoolean` per consentire operazioni di lettura e scrittura atomiche.